Explicit neural-response latency compensation in LANS

27 October 2026 (Asia/Shanghai). Experimental addition; previous frozen studies remain unchanged.

Implemented mechanism

The supervisor now owns an explicit model-based input compensator. At an available PPO query, it forecasts the latest estimated vehicle/actuator state over the anticipated observation-to-response delay and gives the resulting physical features to PPO. On receipt, the ordinary supervisor still evaluates that residual against the latest current-state LQR baseline. The existing default profile is unchanged; this feature is selected explicitly with `delay_compensated`.

$$\hat{d}_i = 0.75\hat{d}_{i-1} + 0.25\,\min(0.04, T_{\mathrm{recv},i} - T_{\mathrm{obs},i}), \qquad \hat{d}_0 = 0.01\ \mathrm{s}.$$

The estimate updates only once for each strictly increasing completed-response sequence. Held packets do not age the estimate repeatedly. It does not read the current configured service delay, pending completion or a future observation. The measured response age includes inference/dispatch/IPC/polling effects; it is distinct from neural inference duration alone.

$$\hat{\xi}(t + \hat{d} \mid t) = F_{\mathrm{nom}}(\hat{\xi}(t), \delta_a(t), \pi_{\mathrm{LQR}}, \mathcal{R}_t, \hat{d}), \qquad \Delta\delta = \pi_{\mathrm{PPO}}(o_t^{\mathrm{pred}}).$$

The forecast uses the nominal bicycle model, present measured speed/gain and reference-curvature preview, with LQR continuation. Euler steps are at most 10 ms, prediction is capped 40 ms, actuator time constant is 50 ms, rate limit 0.35 rad/s and steering-angle limit 0.5 rad. The route preview shifts by speed times prediction horizon. Known policy history is retained; channels 15/16/17/19 remain zero. If the forecast leaves the declared lateral/heading envelope, the actual raw query is used and that reason is logged.

The model interface is separately declared as `latency_compensated_query_v7`. The diagnostic copies frozen v5 NumPy actors without changing their weights. Those weights were not retrained on predicted-state inputs, so distribution shift remains an explicit limitation. Runtime rejects mixing legacy/history and compensated interfaces.

This changes the policy input, unlike freshness-only rejection. Original observation timestamps are never refreshed, and the 50 ms stale cutoff is unchanged. Compensation cannot rescue arbitrarily old packets, reduce physical inference duration, cancel sensor/transport delay, or establish 1,000 ms delayed LQR-feedback tolerance. The existing latest-state cost gate, filter, cap and gradual recovery are unchanged; the earlier proposed multi-weight argmin/first-step-only rollout is not silently substituted.

Controlled development evidence

Eight compensation contracts and seven legacy history/packet checks pass. The fixed diagnostic contains **72 episodes**, **93,600 replayed transitions**, **28,860 submitted actor decisions** and **72/72 survivors**. It compares LQR, raw-query PPO and compensated-query PPO at 20 m/s on one known mirrored path, using three actors/noise groups and 10/20/40/100 ms logical service. No training, tuning, reserved evaluation or promotion occurred.

Logical service	Raw PPO lateral RMSE m	Compensated RMSE m	Tracking gain vs raw	Heading change vs raw	Rate RMS change vs raw
10 ms	0.043041	0.043091	-0.116%	+0.025%	+0.252%
20 ms	0.043725	0.043519	+0.471%	-0.284%	+0.683%

Logical service	Raw PPO lateral RMSE m	Compensated RMSE m	Tracking gain vs raw	Heading change vs raw	Rate RMS change vs raw
40 ms	0.046895	0.046895	+0.000%	+0.000%	+0.000%
100 ms	0.047150	0.047150	+0.000%	+0.000%	+0.000%

All **12** 100 ms-delay raw/compensated-versus-LQR trajectory pairs are exactly equal: compensation does not bypass freshness rejection. Positive table values indicate lower lateral RMSE; negative values indicate harm. This narrow comparison does not establish the earlier full student/fixed-baseline selection criteria or general superiority.

At 40 ms logical service neither neural variant had any nonzero learned authority. A 40 ms-old proposal expires at 50 ms before the unchanged three-good-control-tick recovery can finish; intervening stale decisions reset recovery. Therefore zero difference at 40 ms is baseline-only operation, not evidence of successful 40 ms assisted compensation. The recovery/freshness interaction is preserved and reported rather than bypassed to create a gain.

At 13632 eligible first deliveries on compensated trajectories, raw-versus-current lateral-input RMSE was 0.002841 m and forecast-versus-current RMSE was 0.002219 m. Mean absolute forecast-horizon error was 0.070 ms. These are post-hoc noisy-estimate calibration measurements, not true-state error bounds or a causal proof of tracking improvement.

Real-worker integration

Three actors ran all five actual worker cases for 1 second each: **1500 ticks**, **727 issued forecast queries**, and **712 observable completed-response updates** were replayed. Original deadlines recorded **164 misses**. Input forecasts, known history, source/model hashes, actor outputs, fusion/gate decisions and original stale timestamps were checked. Actual post-inference waits are separate from offline total logical service; their delay distributions are not assumed equal.

This is a short software integration test, not 100 Hz qualification, native Webots confirmation, actual laptop execution or physical sensor/actuator validation. Previous strict real-time failures remain unresolved. A model forecast over estimated delay is approximate compensation; it cannot guarantee cancellation under model mismatch, unexpected stalls, measurement noise or a busy local control process.

The exported Torch-free Python 3.10 kit also passed an independent clean-environment packaging replay: 250 ticks, 34 deadline misses, functional fallback True. This separate short smoke is not pooled with the three-actor integration campaign or presented as actual Windows/macOS execution.

Run and integration

`python tools/reviewer_v7/run7.py --publish` resumes the fixed verified diagnostic without retuning or overwriting partial outcomes. Use the generated actor directory with `python tools/controller_review/laptop_full_controller.py --profile delay_compensated --models docs/research/reviewer-delay-compensation-v7/actor --output laptop-results-compensated` for a separate diagnostic. Existing output folders must not be overwritten.

The new experimental laptop ZIP uses the same profile and explicit model declaration. Extract it before running RUN_WINDOWS.cmd or `bash RUN_MAC.command`. This does not make a selected/deployment controller. Keep raw results even if prediction harms performance.

The diagram can show a Model-based Delay Compensator inside the supervisor, taking past completed-response timing and current estimated state/reference, and supplying forecast physical features to the Observation Builder/PPO. The admission gate must retain its separate latest-state input.

The honest report wording is: **LANS includes experimental causal model-based compensation for anticipated neural-response latency, together with latest-state predictive admission, unchanged stale-proposal rejection and LQR fallback.** Physical command-delay cancellation and guaranteed real-time operation are not claimed.